Python Programming Language — Official Course Syllabus

Course Code: CS-204 | Academic Year: 2026 | Total Credits: 4.0 | Evaluation: 100 Marks

Unit 1: Python Fundamentals, Syntax, and Control Structures

Weightage: 20% | Estimated Study Time: 12 Hours

This foundational unit covers core language architecture, execution environments, and basic programming primitives:

- Python Interpreter execution model, Bytecode (.pyc) compilation, and the CPython Virtual Machine.
- Primitive Data Types: Integers, Floats, Booleans, Strings, Complex numbers, and type casting mechanisms.
- Operators: Arithmetic, Comparison, Logical, Bitwise, Assignment, and Identity (`is` vs `==`).
- Control Flow Statements: if-elif-else branching, nested conditionals, ternary expressions.
- Iteration Constructs: while loops, for loops with range(), break, continue, and loop-else clauses.
- Built-in Functions and I/O: print() formatting (f-strings, format, % specifiers), input(), type(), id(), len(), dir().

Unit 2: Data Structures and Functional Programming Primitives

Weightage: 20% | Estimated Study Time: 14 Hours

In-depth exploration of core container collections, memory mutability, and functional idioms:

- Sequences: Lists (slicing, indexing, in-place mutating methods vs non-mutating operations, sorting algorithms).
- Immutability and Tuples: Packing, unpacking, named tuples, hashability criteria, and performance advantages.
- Key-Value Mapping: Dictionaries, hash table collisions, dict views (keys, values, items), and default dictionary patterns.
- Mathematical Collections: Sets, frozensets, union, intersection, difference, symmetric difference, subset evaluation.
- Functional Expressions: List comprehensions, dictionary comprehensions, set comprehensions, nested comprehensions.
- Higher-Order Functions: Anonymous lambda functions, map(), filter(), functools.reduce(), zip(), and enumerate().

Unit 3: Object-Oriented Programming (OOP) and Exception Handling

Weightage: 25% | Estimated Study Time: 16 Hours

Comprehensive study of class design, polymorphism, and robust error management:

- Classes and Instances: Instance variables, class variables, `self` reference, constructor `__init__`, destructor `__del__`.
- Encapsulation: Public, protected (`_`), and private (`__`) attributes, name mangling, `@property` getters and setters.

- Inheritance Hierarchies: Single, multiple, and multilevel inheritance, Method Resolution Order (MRO), C3 linearization, super().
- Polymorphism and Duck Typing: Operator overloading via dunder methods (`__str__`, `__repr__`, `__add__`, `__eq__`, `__len__`).
- Exception Lifecycle: try, except, else, finally blocks, catching specific exception hierarchies (ValueError, TypeError, KeyError).
- Custom Exceptions: Inheriting from built-in `Exception`, re-raising with `raise`, exception chaining with `from`.

Unit 4: File I/O, Modules, and the Python Standard Library

Weightage: 20% | Estimated Study Time: 12 Hours

Interfacing with filesystem resources, modular architecture, and structured data handling:

- File Operations: Reading, writing, appending text and binary files, buffer flushing, file pointers with seek() and tell().
- Context Managers: The `with` statement protocol, deterministic cleanup, implementing `__enter__` and `__exit__`.
- Modularization: Importing modules, packages, `__init__.py`, namespace isolation, `if __name__ == '__main__':` paradigm.
- Serialization & Formats: Parsing and dumping JSON with `json` module, tabular handling with `csv` module, pickle security cautions.
- System Utilities: Directory navigation and environment variables with `os` and `pathlib`, command-line arguments with `sys.argv`.

Unit 5: Advanced Python: Generators, Decorators, and Concurrency Models

Weightage: 15% | Estimated Study Time: 12 Hours

Advanced engineering patterns, memory-efficient streams, and execution optimization:

- Iterators and Generators: Iterable protocol (`__iter__`, `__next__`), `yield` expressions, generator functions vs generator expressions.
- Function Decorators: Closures, first-class functions, parameter-accepting decorators, functools.wraps preservation.
- Concurrency Architecture: Global Interpreter Lock (GIL) implications, CPU-bound vs I/O-bound bottlenecks.
- Multithreading vs Multiprocessing: `threading.Thread` vs `multiprocessing.Process`, shared state vs process isolation.
- Package Management: pip, requirements.txt, virtual environments (venv), and PyPI distribution basics.